```
template<typename T>
class lock_free_stack
{
private:
    struct node;

    struct counted_node_ptr     <-(1)
    {
        int external_count;
        node* ptr;
    };

    struct node
    {
        std::shared_ptr<T> data;                    (2)
        std::atomic<int> internal_count;      <-(2)
        counted_node_ptr next;           <-(3)


        node(T const& data_):
            data(std::make_shared<T>(data_)),
            internal_count(0)
        {}
    };

    std::atomic<counted_node_ptr> head;     <-(4)

public:
    ~lock_free_stack()
    {
        while(pop());
    }

    void push(T const& data)     <-(5)
    {
        counted_node_ptr new_node;
        new_node.ptr=new node(data);
        new_node.external_count=1;
        new_node.ptr->next=head.load();
        while(!head.compare_exchange_weak(new_node.ptr->next,new_node));
    }
};
```

First, the external count is wrapped together with the node pointer in the `counted_node_ptr` structure ❶. This can then be used for the `next` pointer in the `node` structure ❸ alongside the internal count ❷. Because `counted_node_ptr` is just a simple `struct`, you can use it with the `std::atomic<>` template for the `head` of the list ❹.